Technische Universitaet Clausthal

Department of Informatics

Benchmarking LLM-Generated Web Interfaces

A Reproducible Multi-Metric Leaderboard Framework

Master's Thesis

Xinyang Xie

June 3, 2026

Contents

Chapter 1

Introduction

This chapter will introduce the motivation, research context, and contribution of the thesis. It is intentionally kept as a short placeholder so that the following draft chapters keep their intended numbering while the full introduction is developed.

Chapter 2

Background and Related Work

2.1 Research Gap

The preceding sections have reviewed GUI quality assessment, LLM-generated interfaces, LLM-as-a-judge methodology as an evaluation tool, and multimodal web agents. Table 2.1 places the most relevant prior works along seven dimensions that together characterise a comprehensive GUI quality evaluation framework. Taken together, three observations motivate the present thesis.

Existing website-generation benchmarks do not directly answer generator-leaderboard questions. UIClip [8] demonstrated that vision-language models can perform pairwise UI comparison, but it relies on models that predate the current generation of multimodal LLMs and limits evaluation to a binary preference. The LLM-as-a-judge literature [10, 4] has established scalable evaluation methods for text, yet those methods also introduce known bias and calibration concerns. In this thesis they are therefore treated as practical scoring instruments rather than as the main leaderboard target. Vision2Web [1] moves closer to this thesis by evaluating visual website development with both visual judging and agent-based verification, but its primary object is coding-agent website development rather than a thesis-specific, multi-metric leaderboard for comparing generated GUI submissions across static and dynamic metrics.

Requirement fidelity and rubric-based assessment are largely absent from GUI evaluation. Design2Code [7] evaluates how faithfully generated code reproduces a visual reference, but does not assess richer quality dimensions such as visual hierarchy, information clarity, or perceived usability. The Designer Feedback study [9] found that simple pairwise rankings among evaluators agreed at only around 50%, suggesting that coarse binary judgements are insufficient for reliable GUI assessment. A structured rubric grounded in established usability theory [5, 2] can provide finer-grained and more

reproducible quality signals.

Static visual metrics have rarely been related to dynamic interaction outcomes. VisualWebArena [3] and WebArena [11] evaluate multimodal agents on goal-directed web tasks, but do not assess the GUI quality of the interfaces themselves. How static quality scores relate to task completion, interaction feedback, and functional behaviour remains an open empirical question. Answering this question requires a benchmark that keeps generated interfaces executable so that static and dynamic evaluations can be performed on the same set of items.

The present thesis addresses all three gaps simultaneously by centring the benchmark on controlled generation of executable web interfaces, standardized artifact scoring, model-level aggregation, and a static-plus-dynamic leaderboard for comparing generator LLMs. UIClip-style pairwise judgement is retained only as a small baseline or sanity check. LLM-based scorers and human review are treated as scoring instruments and audit signals rather than as the main leaderboard target.

Table 2.1: Comparison of related work across seven evaluation dimensions. ✓ = fully addressed; △ = partially addressed; × = not addressed; — = not applicable to this line of work.

Work	**Multi-dimensional rubric**	**Generated GUI quality**	**Bias and reliability analysis**	**Human reference labels**	**Requirement fidelity**	**Dynamic interaction eval**	**Benchmark artifact**
UIClip [8]	×	×	×	✓	×	×	×
Designer Feedback [9]	×	✓	×	✓	△	×	×
Design2Code [7]	△	✓	×	✓	✓	×	✓
Vision2Web [1]	△	✓	△	△	✓	✓	✓
MT-Bench / Chatbot Arena [10]	△	—	△	✓	—	—	△
LLM-as-a-Judge survey [4]	✓	—	✓	—	—	—	×
Position Bias Study [6]	×	—	✓	✓	—	—	×
WebArena [11]	×	△	×	✓	△	✓	✓
VisualWebArena [3]	×	✓	×	✓	△	✓	✓
This thesis	✓	✓	△	△	✓	✓	✓

Chapter 3

Benchmark Design and Methodology

This chapter defines the benchmark and evaluation protocol used in this thesis. The central object being scored is the generated UI or Web app artifact. The leaderboard ranking target is the generator LLM: artifact-level scores are aggregated across a fixed benchmark item set into model-level leaderboard rows.

The chapter first states the research questions, then defines the benchmark object model, generation protocol, rendering protocol, metric categories, dynamic validation strategy, reliability audit, and the mapping from research questions to methods.

3.1 Research Questions

The thesis investigates how to evaluate LLM-generated web interfaces with a reproducible benchmark and leaderboard. It uses automated scoring, human review, and LLM-based judging as measurement instruments, but these instruments are not the main leaderboard target.

RQ1 – Benchmark and leaderboard design.

> *How can a reproducible benchmark and leaderboard be designed for evaluating LLM-generated web interfaces?*

RQ1 defines the fixed benchmark dataset, benchmark item schema, generation protocol, generated artifact format, metadata requirements, and aggregation from artifact-level scores to generator-model leaderboard rows. Track A/UIClip-style material is retained only as a small baseline or sanity check, not as the main research question or primary contribution.

RQ2 – Metric operationalization and aggregation.

How can static technical, static visual, dynamic task-based, and efficiency metrics be operationalized and aggregated for generated web apps?

RQ2 defines the category scores used by the leaderboard: Static Technical Score, Static Visual Score, Dynamic Score, and Efficiency Score. An optional Overall Score may be reported, but category-specific rankings remain visible because different users may prioritize visual quality, functional correctness, or cost efficiency differently.

RQ3 – Static and dynamic quality relationship.

How are static quality scores related to dynamic task-success outcomes in LLM-generated web apps?

RQ3 evaluates the same generated artifacts with both static and dynamic signals. The analysis reports correlations and mismatch cases, such as visually strong interfaces that fail tasks or visually plain interfaces that complete required workflows reliably.

Reliability audit.

How stable and bias-sensitive are the LLM-based visual or rubric scores used in the evaluation pipeline?

The reliability audit supports the scoring protocol. It records repeated scoring consistency, order-swap sensitivity for pairwise comparisons when used, rating-scale sensitivity if feasible, judge-model variance, and invalid or refusal rates. It is not a separate main research question and does not make judge-model comparison the thesis target.

3.2 Overall Benchmark Design

The benchmark is based on fixed items and controlled generation. A generator model receives the same benchmark context and standardized prompt schema as other models in the same comparison batch. The model produces executable web artifacts, and those artifacts are scored with the same evaluation protocol.

Direct evaluation target. The generated UI/Web app artifact is scored for one benchmark item. Scores include technical coverage, visual quality, dynamic task behavior, and efficiency metadata.

Leaderboard ranking target. The generator LLM or generator model configuration is ranked. Artifact-level scores are aggregated across benchmark items into model-level rows.

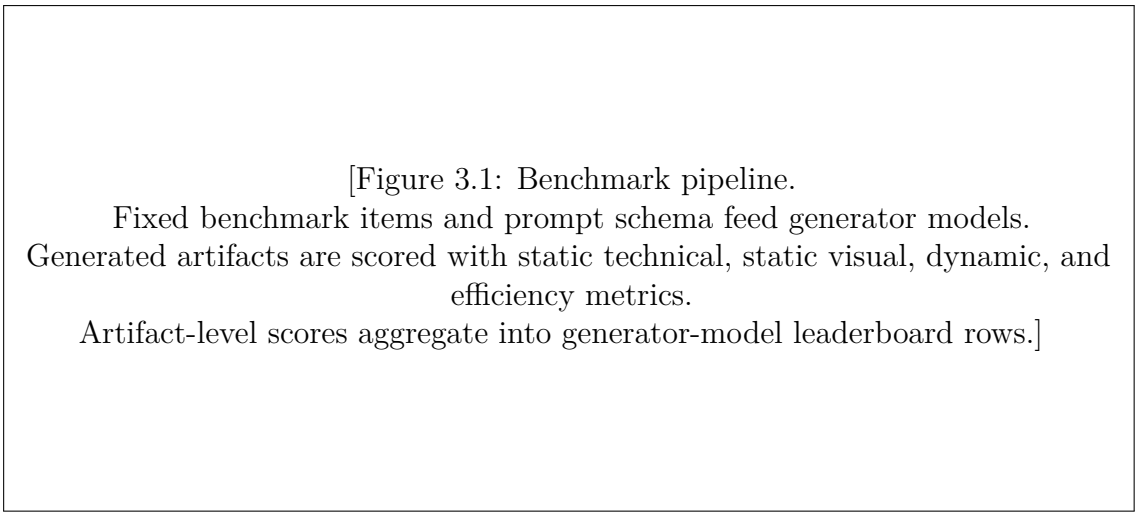

Figure 3.1: Benchmark and leaderboard pipeline. Generated artifacts are the direct scoring target; generator models are the leaderboard ranking target.

Primary mode. The main mode is New LLM mode: a generator model API/interface is added to the pipeline, the fixed benchmark set is run, and the result is a model row in the leaderboard.

Secondary mode. A single HTML file or zipped project may be scored as an artifact-level report, but this is secondary. It is not the main leaderboard basis unless the artifact was generated under the fixed benchmark protocol and includes complete model, prompt, item, timestamp, and generation metadata.

3.3 Benchmark Item and Generated Artifact

A benchmark item is the fixed test specification. It is not a generated output. Each item contains the information needed to generate and evaluate a web interface:

- natural-language requirement text;
- expected pages or routes;
- required elements, text, controls, forms, and selectors;
- dynamic tasks or workflows;
- validation rules;
- predefined pages or routes for static visual screenshots;
- metadata such as source, category, difficulty, and whether dynamic validation is required.

The recommended planning schema is:

```
benchmark_item/
  requirement.md
  pages.json
  elements.json
  tasks.json
  validation_rules.json
  visual_pages.json
  metadata.json
```

A generated artifact is the output produced by one generator model for one benchmark item. The minimal artifact is an executable web implementation plus generation metadata. The metadata records model identity, provider, prompt ID, timestamp, provider parameters, token usage, latency, cost, and finish reason.

3.4 Dataset Tracks

3.4.1 Track A: Baseline or Sanity Check

Track A uses a reduced UIClip-style screenshot baseline. Its purpose is to anchor the study in prior screenshot-based GUI evaluation and to provide a sanity check for pairwise judging. It is not the primary contribution and does not define the main leaderboard objective.

Track A can reuse existing UIClip preference labels for a small sample of screenshots. No new large annotation campaign is required for this baseline.

3.4.2 Track B: Generated Web Interfaces

Track B is the main benchmark track. It uses requirement-driven web interface generation tasks, primarily from a reduced Vision2Web Level 1/Level 2 subset. Level 3 full-stack tasks are excluded because they introduce backend state, deployment, and long-horizon engineering requirements that are outside the current benchmark scope.

The development prototype should begin with 2–5 benchmark web app items. If feasible, the thesis experiment may expand to 10–20 benchmark tasks. Expansion depends on generation cost, scoring cost, rendering reliability, and the amount of usable dynamic validation available in the selected items.

3.5 Generation Protocol

For direct generator comparisons, all models in a comparison batch receive the same benchmark item context and the same standardized prompt schema. Model- specific

prompt tuning is not allowed within a batch because it would confound generator quality with prompt engineering.

The main input to the leaderboard pipeline is a generator model API/interface. For each model, the pipeline loads the fixed benchmark items, applies the standardized prompt, saves generated artifacts, records generation metadata, and runs the evaluation protocol.

The evaluation should avoid presenting any fixed truncating output-token cap as the final policy. If a provider requires a maximum output setting, it should be set high enough to allow complete outputs where feasible. Actual input tokens, output tokens, total tokens, finish reason, cost, and latency are recorded as efficiency and reproducibility data.

Prompts that affect thesis results, metric computation, benchmark protocol, human annotation, output schemas, or reproducibility are versioned and preserved according to the prompt preservation policy.

3.6 Rendering and Screenshot Protocol

Generated artifacts are rendered locally with a controlled browser environment. Rendering produces the evidence used by later scoring modules.

For static visual scoring, the pipeline visits only the predefined pages or routes specified by the benchmark item and captures standardized screenshots. The browser engine, viewport, zoom level, wait rule, and screenshot mode are kept fixed across artifacts. These screenshots are the input to static visual evaluation.

Agent trace screenshots are not used as the main static visual scoring input. They may be stored for debugging and dynamic failure analysis only, because agent traces can vary in length, path, and content across generated apps.

Rendering failures such as blank pages, fatal JavaScript errors that prevent visible content, or missing entry files are recorded as artifact-level failures.

3.7 Metric Categories

3.7.1 Static Technical Score

Static technical metrics evaluate structure and requirement coverage without executing a user task. Candidate submetrics include HTML completeness, required element coverage, required text coverage, route declaration coverage, link target declaration, form field coverage, selector or ID coverage, accessibility basics, and static requirement fidelity.

Static technical checks can verify that a required button exists and declares a target.

They do not prove that clicking the button completes a task; that is a dynamic metric.

3.7.2 Static Visual Score

Static visual metrics evaluate standardized predefined-route screenshots. Candidate visual dimensions include visual hierarchy, information organization, typography and readability, contrast, spacing and alignment, consistency, and overall aesthetic quality.

Page-level visual scores are aggregated into an artifact-level Static Visual Score. Agent trace screenshots are excluded from the main static visual score.

3.7.3 Dynamic Score

Dynamic metrics evaluate interaction behavior. A metric is dynamic when it requires clicking, typing, navigating, submitting, waiting for state changes, or validating task completion.

Candidate submetrics include task success rate, step success rate, route execution success, button functionality success, form completion success, state-change success, retry count, failure localization, and robustness or pass@k.

The current implementation separates the dynamic metric schema from the executor. Early pilot runs may use deterministic route simulation, while the browser workflow evaluator executes the same workflow cases in Chromium with Playwright, clicks generated controls, and validates visible destination content. A future implementation may replace this evaluator with a real computer-use agent, but the metric schema should remain stable: change the executor, not the metric schema.

3.7.4 Efficiency Score

Efficiency metrics track input tokens, output tokens, total tokens, generation cost, evaluation cost, generation latency, evaluation latency, and cost-performance trade-offs.

Efficiency is reported both as raw metadata and, when a reference is defined, as a normalized score. Cost-performance plots and Pareto frontiers are used to show whether a model is dominated by another model with both lower cost and higher quality.

3.7.5 Optional Overall Score

The initial optional composite score is:

```
Overall Score =
0.25 * Static Technical Score
+ 0.25 * Static Visual Score
+ 0.35 * Dynamic Score
```

```
+ 0.15 * Efficiency Score
```

This composite is a reporting convenience. Category-specific scores remain visible and are interpreted separately.

3.8 Dynamic Validation Strategy

The thesis first implements workflow-based dynamic validation rather than heavy real-agent execution. This choice keeps the pipeline reproducible and avoids confounding generated-interface quality with the planning or grounding failures of a separate agent.

For Vision2Web-derived Level 2 items, dynamic tasks can be derived from workflow actions and validations. The evaluator checks whether the generated artifact supports the expected route, form, state, or content outcome. The current pipeline distinguishes deterministic route simulation from browser-workflow validation. Route simulation reads the generated artifact and checks route/content evidence without driving a real browser. Browser-workflow validation opens the generated HTML in Chromium, executes workflow clicks with Playwright, records route success, and validates visible destination content. Both are dynamic validation signals, but neither is presented as a full real-agent usability study.

Real-agent execution remains an optional future upgrade or small cross-check after the main pipeline is stable. If added later, the same task success and failure taxonomy should be reused so that only the executor changes.

3.9 Failure Taxonomy

Failures are recorded so that leaderboard scores remain interpretable:

- **Generation failure**: no artifact, provider timeout, invalid response, or truncated unusable output.
- **Render failure**: artifact exists but cannot be rendered into visible content.
- **Static technical failure**: required structure, route, element, text, selector, or basic accessibility evidence is missing.
- **Static visual failure**: standardized screenshot shows severe visual defects such as overlap, unreadable text, or unusable layout.
- **Dynamic failure**: required task, route, form, button, or state change fails under the dynamic validation protocol.

Table 3.1: Research question – method mapping.

RQ	Question	Data	Method	Primary outputs
1	Benchmark and leaderboard design	Fixed benchmark items; generated artifacts	New LLM mode; artifact scoring; model-level aggregation	Benchmark schema; prompt protocol; leaderboard row format
2	Metric operationalization and aggregation	Generated artifacts; screenshots; task validations; metadata	Static technical, static visual, dynamic, and efficiency scoring	Category scores; optional Overall Score; cost-performance plots
3	Static and dynamic quality relationship	Same Track B artifacts scored statically and dynamically	Correlation analysis and mismatch case analysis	Static-dynamic correlations; qualitative divergence cases
Audit	Scoring reliability	Selected scoring outputs	Repeated scoring, order-swap checks, variance and invalid-rate logs	Reliability and bias audit results

- **Evaluator/tool failure**: scoring or validation tool fails for reasons unrelated to the generated artifact.

Evaluator or tool failures are reported separately from generated-artifact quality when possible.

3.10 Reliability Audit

LLM-based visual or rubric scoring can be affected by prompt sensitivity, position bias, scale effects, model variance, and invalid outputs. These issues are audited but are not the main thesis target.

The audit may include repeated scoring consistency, order-swapped pairwise comparisons where pairwise scoring is used, rating-scale sensitivity if feasible, judge model variance, and invalid or refusal rate. Audit results are reported as scoring reliability evidence and threats to validity.

3.11 Research Question – Method Map

Table 3.1 maps the research questions to data, methods, metrics, and outputs.

Bibliography

[1] Zehai He, Wenyi Hong, Zhen Yang, et al. Vision2Web: A hierarchical benchmark for visual website development with agent verification. *arXiv preprint arXiv:2603.26648*, 2026.

[2] International Organization for Standardization. ISO 9241-11:2018 — ergonomics of human-system interaction: Usability: Definitions and concepts. International Standard 9241-11, ISO, 2018.

[3] Jing Yu Koh, Robert Lo, Lawrence Yunliang Chen, et al. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proc. ACL*, 2024.

[4] Dawei Li, Bohan Jiang, Liangjie Huang, et al. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proc. EMNLP*, 2025.

[5] Jakob Nielsen and Rolf Molich. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 249–256. ACM, 1990.

[6] Lin Shi, Chiyu Ma, Wenhua Liang, et al. Judging the judges: A systematic study of position bias in LLM-as-a-judge. In *Proc. IJCNLP-AACL*, 2025.

[7] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, et al. Design2Code: How far are we from automating front-end engineering? In *Proc. ACL*, 2024.

[8] Jason Wu, Yi-Hao Peng, Xin Yue Amanda Li, et al. UIClip: A data-driven model for assessing user interface design. In *Proc. ACM UIST*, 2024.

[9] Jason Wu, Amanda Swearngin, Arun Krishna Vajjala, et al. Improving user interface generation models from designer feedback. In *Proc. CHI*, 2026.

[10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2023.

[11] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. WebArena: A realistic web environment for building autonomous agents. In *Proc. ICLR*, 2024.

Appendix A

Prompt Templates and Experimental Metadata

This appendix records prompts that materially affect the thesis methodology, experimental results, benchmark protocol, or reproducibility. Routine prompts used for coding, brainstorming, language polishing, or project management are not included unless their outputs become part of the experimental protocol.

A.1 Inclusion Rule

A prompt is included when changing or omitting it could change a reported metric, model comparison, benchmark construction step, annotation protocol, or reproducibility claim. For each included prompt, the experiment records the prompt text, model identifier, provider, date, key decoding parameters, input modalities, output format, random seed if applicable, and source result file.

A.2 Track A Pairwise Judgement Prompts

A.2.1 TA-PAIR-ZS-v1: Zero-shot Pairwise Visual Design Prompt

Purpose: Baseline pairwise GUI screenshot judgement for Track A, including the order-swapped reliability condition. The model receives two screenshots labelled Image A and Image B and returns only the preferred letter.

Metadata:

- **Track / task:** Track A, pairwise preference judgement
- **Prompt ID:** TA-PAIR-ZS-v1

- **Models used:** `gpt-4o` (OpenAI); `claude-sonnet-4-5-20250929` (chatanywhere-anthropic proxy)
- **Input modalities:** two PNG screenshots (Image A, Image B) + user-role text
- **Message structure:** single user message; no system message
- **Decoding parameters:** temperature = 0, max_tokens = 5
- **Seed:** 42 (pair sampling); model temperature is 0 so output is deterministic
- **Order-swap:** applied; each pair is evaluated in both AB and BA orders
- **Dataset:** `biglab/uiclip_human_data_hf` (HuggingFace), train split
- **Source:** `scripts/track_a_eval.py`, lines 44–52 (`ZEROSHOT_PROMPT`)
- **First use:** 2026-05-06 (gpt-4o pilot); 2026-05-19 (claude-sonnet-4-5 run)
- **Result files:** `scripts/results/track_a_eval_gpt-4o_20260506_*`; `scripts/results/track_a_eval_chatanywhere-anthropic_claude-sonnet-4-5-20250929_`

```
You are evaluating two user interface screenshots for visual design quality.

Image A is shown first, Image B is shown second.

Which interface has better visual design overall? Consider layout, visual
hierarchy, clarity, and aesthetic quality.

Reply with ONLY the letter A or B. Nothing else.
```

A.2.2 TA-PAIR-RUBRIC-v1: Rubric-guided Pairwise Visual Design Prompt

Purpose: Rubric-guided Track A pairwise GUI screenshot judgement. Adds four explicit design dimensions before requiring the same A/B output format as the zero-shot condition. Used in the prompting-strategy ablation.

Metadata:

- **Track / task:** Track A, pairwise preference judgement (rubric condition)
- **Prompt ID:** TA-PAIR-RUBRIC-v1
- **Models used:** `gpt-4o` (OpenAI)
- **Input modalities:** two PNG screenshots (Image A, Image B) + user-role text
- **Message structure:** single user message; no system message

- **Decoding parameters:** temperature = 0, max_tokens = 5
- **Seed:** 42 (pair sampling)
- **Order-swap:** applied
- **Dataset:** `biglab/uiclip_human_data_hf` (HuggingFace), train split
- **Source:** `scripts/track_a_eval.py`, lines 54–68 (`RUBRIC_PROMPT`)
- **First use:** 2026-05-19
- **Result files:** `scripts/results/track_a_eval_gpt-4o_20260506_142748_pair_review/`

```
You are an expert UI design reviewer evaluating two user interface screenshots.
Image A is shown first, Image B is shown second.

Score each interface on the following four dimensions before deciding:

1. Visual structure - layout, alignment, spacing, and visual hierarchy.
2. Information clarity - readability, labelling, and grouping of related items.
3. Aesthetic quality - typography, colour use, and overall polish.
4. Inferred usability - how easy the interface looks to use at a glance.

Briefly compare the two interfaces dimension by dimension in your head, then
pick the one that is better overall.

Reply with ONLY the letter A or B on the last line. Nothing else.
```

A.3 Track B GUI Generation Prompts

A.3.1 Track B Generation Prompt Version History

Track B generation prompts changed repeatedly during smoke testing. To keep the appendix focused on reproducibility rather than serving as a full prompt archive, this appendix reproduces in full only the frozen baseline prompt that produces the reported Track B pilot and leaderboard artifacts, `TB-GEN-v9`. All earlier and later smoke-test variants (`TB-GEN-v1` through `v8`, `v12`, and `v13`) were diagnostic iterations that did not contribute reported metrics; they are summarized in Table A.1 and remain recoverable in full from the source template in `scripts/generate_track_b_ui.py`, per-run `generation_metadata.json` files, and the revision log when needed to interpret a specific pilot artifact.

A.3.2 TB-GEN-v9: Per-Item Workflow-Label Generation Prompt

Purpose: Practical Track B baseline prompt for the cross-item smoke batch. This version generalizes the v8 workflow-control idea by filling the workflow label list auto-

matically from each benchmark item's workflow file rather than using a hand-written item-specific mapping.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v9
- **Version change:** revises TB-GEN-v8 by replacing the hard-coded F09 workflow-control list with an automatically extracted `{workflow_labels}` slot
- **Models used:** `qwen3.6-35b-a3b` via GWDG/SAIA OpenAI-compatible API
- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** temperature = 0.2, max_tokens = 20000 in smoke tests
- **Dataset:** `zai-org/Vision2Web`, reduced Level 1 / Level 2 pilot subset
- **Source:** `scripts/generate_track_b_ui.py`
- **First use:** 2026-05-26, Europe/Berlin
- **Result files:** `data/track_b/items/F01_1daycloud/generated/gwdg_qwen36_35b_v9_smoke` `data/track_b/items/F03_about_gitlab/generated/gwdg_qwen36_35b_v9_smoke/`; `data/track_b/items/F10_gourmania/generated/gwdg_qwen36_35b_v9_smoke/`

```
System:
You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:
Prompt ID: TB-GEN-v9

Build a minimal complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:
- Return exactly one complete HTML document ending with </html>.
- Include inline CSS and JavaScript only. No build step, backend, network access, external CDN, or markdown.
- Local assets must be referenced as ../../resources/<subpath>.
- Use the prototype screenshots for overall structure and visual cues, but produce a minimal benchmark implementation.
- Primary viewport is desktop: 1365px and 1920px. Mobile quality is not a hard gate.
- Implement every route in the route contract as a <section id="..." data-track-route>.
- Include window.__TRACK_B_ROUTES exactly as given and a showPage(routeId) handler before </body>.

Mandatory workflow controls:
- The following exact workflow click labels were extracted from the workflow checks. Every label below must appear as vis
{workflow_labels}
```

```
- Never use <div>, <span>, <li>, or other inert elements for workflow-required click labels.
- Every workflow control must include data-route-target="<route_id>" and onclick="showPage('<route_id>')".
- If a click target is a feature phrase such as "GitLab Duo", use that feature phrase as the visible label.
- Choose the route target from the route contract that best matches the label and workflow validation.
- Secondary navigation items may be visible non-functional placeholders only if they are not listed in the extracted work

Strict compactness limits:
- Use one shared header and one shared footer only.
- For each route, use at most one h1, one short paragraph, one compact card/list, and one optional tiny table.
- Use at most 3 cards per route, 3 bullets per list, and 3 rows per table.
- Do not copy long requirement text. Use short labels and short summaries.
- Avoid decorative animations, large CSS frameworks, inline SVG art, exhaustive menus, and long repeated sections.
- If output budget feels tight, remove visual detail first. Never omit routes, workflow controls, JavaScript, or closing

Route contract:
{route_contract}

Required JavaScript:
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
  document.querySelectorAll('[data-route-target]').forEach(function(link) {
    link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
  });
  if (history.replaceState) history.replaceState(null, '', '#' + routeId);
  setTimeout(function() { window.scrollTo(0, 0); }, 0);
}
document.addEventListener('DOMContentLoaded', function() {
  var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
  showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>

Item ID: {item_id}
Source task: {source_task_name}
Source level: {source_level}
Use for dynamic validation: {use_for_dynamic}

Normalized requirement:
{requirement}

Workflow checks:
{workflow}

Available local resource paths:
{resource_paths}

Prototype screenshots are attached after this text in this order:
{prototype_list}
```

A.4 Prompt Slots for Later Experiments

Future thesis-impacting prompts should be added here before or immediately after their first experimental use. Expected prompt families include Track B static rubric scoring prompts, LLM-based dynamic validation prompts if such validators are later used, output-schema prompts, and human annotation instructions.

Prompt ID	Use	Main change	Status in appendix
`TB-GEN-v1`	First smoke run	Baseline single-file HTML generation prompt for normalized Vision2Web pilot items.	Summarized only
`TB-GEN-v2`	Smoke iteration	Added explicit completion, same-file JavaScript handlers, workflow-action non-placeholder requirements, and responsive-layout constraints.	Summarized only
`TB-GEN-v3-v4`	Intermediate iteration	Refined workflow route handling before the fixed route-contract version.	Summarized only
`TB-GEN-v5`	Route-contract smoke run	Added a fixed route contract, a minimal route handler, no-scroll hash replacement, and stricter mobile overflow safeguards.	Summarized only
`TB-GEN-v6`	Reproducible smoke template	Made desktop workflow fidelity the hard generation target and clarified handling of unavailable downloadable assets.	Summarized only
`TB-GEN-v7`	Compact same-prompt comparison template	Keeps the same route/workflow contract as v6 but instructs models to produce a shorter complete implementation to reduce truncation and gateway-timeout risk.	Summarized only
`TB-GEN-v8`	Compact gated comparison template	Adds explicit exact workflow-label-to-route mappings and forbids inert elements for workflow-required controls after v7 produced an inert Local Forms control.	Summarized only
`TB-GEN-v9`	Per-item workflow-label prompt (frozen baseline)	Generalizes v8 by automatically extracting workflow click labels from each item's workflow file instead of hard-coding the F09 label-to-route list. Frozen as the practical baseline for the cross-item pilot batch.	Full text below
`TB-GEN-v12`	Machine-checkable workflow-control contract	Adds per-item `visible_text`, `route_id`, and `required_element` entries to reduce ambiguity around exact clickable labels; did not resolve the exact-label case and was not adopted.	Summarized only
`TB-GEN-v13`	Required workflow-control navigation block	Replaces per-control examples with a complete navigation block to copy into the page header; both smoke attempts failed at the provider layer (HTTP 500) and produced no artifact.	Summarized only

Table A.1: Summary of Track B GUI generation prompt versions. Only the current templates are reproduced in full in this appendix; prior versions are retained through experiment artifacts and source history.